* Constructor Chaining: → (Placement Oriented Concept)

Benefits:
(i) No code duplication
(ii) More efficient code
(iii) Better readability

It is done in the same class by using the this keyword

this( ) → Current Class
Super( ) → Parent Class

Constructor

* Interface is used to solve the diamond problem.
* Interface methods cannot have a body (partial)
* [Latest Java allows static & default methods for interfaces]

Strings in Java: →

* A string is a sequence of characters. It can contain:
  a) Numbers (0 – 9)
  b) Characters (A – Z or a – z)
  c) Special symbols (*, –, +, &, %)
  d) Whitespaces " " .
* In Java String is a class.
* It belongs to the package [java.lang] *** It is auto imported
* String are immutable (They cannot be changed)

Immutable means → once a String object is created, it cannot be changed.
If you "modify" it, Java will create a new object. (Heap Memory)

For example:

```
String s = "Java";
       s = s + "DSA";  [concatenation]
```

"Java" → Object 1 → Heap
"Java DSA" → Object 2 → Heap

* Object 1 becomes garbage.

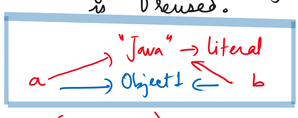
So, any modification = new Object.

SCP [String Constant Pool]

* Stored inside Heap Memory.
* Special area called the String Constant Pool
* If same literals (values), reference or object is reused.

```
String a = "Java";
String b = "Java";
```

SCP →



Both refer to the same object.

But,

```
String a = new String("Java");
String b = new String("Java");
```

Two different objects.

IMMUTABLE

```
String s = "Java" → object 1 → hashcode → 1234
s + " DSA";       → object 2 → hashcode → 0236
```

modification

MUTABLE

```
StringBuilder builder = new StringBuilder("Java");
builder.append(" DSA");
```

builder → hashcode = 1234
builder.append(" DSA"); hashcode = 1234

Object same